Daily Workflow

One interactive session owns a ticket through eight phases: brainstorm, spec, plan, implement, tests, docs, review, done. Every phase but the light one below writes a file under `.work/tickets/<id>/`, and two of them cannot be skipped by accident — `/crew:implement` refuses without an approved plan, and `/crew:done` refuses without a clean review receipt, a verify gate settled for HEAD, and a passing completion audit. Settled means every rule passed here, or the `verify-gate` CI receipt for HEAD is VERIFIED (for a merge on main, the PR head's receipt when its tree is exactly the merge's): a rule too big for the Stop budget is deferred to CI, and a Stop turn where 0 rules ran proves nothing.

For how the scope guard decides what a ticket may touch and what a report looks like when it refuses a write, see `daily-workflow-scope.md` (T3).

Full Path — Worked Example

1. Brainstorm. You type:

```
/crew:brainstorm add a --dry-run flag to the migrate command
```

The session mints `T-0091`, creates `.work/tickets/T-0091/`, and asks **one question at a time**:

> Should `--dry-run` also print what it would do to files that already match (no-op writes), or only the ones that would actually change?

You answer. It proposes two approaches, recommendation first, then writes `.work/tickets/T-0091/direction.md` and stops. You say yes.

2. Spec. You type `/crew:spec T-0091`. The session reads the direction, uses `crew:explorer` to pin down `path:line` evidence, and writes `spec.md` with Intent, Exclusions, Evidence, Unknowns, Touch and Acceptance. `.work/INDEX.md` gets a row.

If the ticket cannot start before others close, the spec carries an optional `depends-on: [T-0088, T-0090]` line under its header. `blocked` is then derived from it, never typed: any dependency that is open, `cancelled`, `superseded` or cannot be read blocks, and `done` or `merged` unblocks

(`hooks/scripts/crew_ticket_state.py`). The line is hashed with the spec, so changing it needs `/crew:approve` again. `needs-replan` is derived the same way, from the review ledger.

3. Plan. You type `/crew:plan T-0091`. The session reads `spec.md`, writes one step per unit of work in `plan.md` (Files/Test/Risk/Standards each), checks every Files: entry against the spec's Touch globs, and enters **plan mode** to show you the whole thing. You review it. On your yes, it asks you to type `/crew:approve T-0091`. crew's prompt hook sees that you typed it and writes the approval receipt, bound to this plan's exact contents — edit `plan.md` after this and the receipt no longer matches.

4. Implement. You type `/crew:implement T-0091`. First thing it does: checks the receipt. If you skipped step 3, or edited the plan after approving it, it refuses here and tells you which. Assuming it passes, it records the scope base (`scope_base.py --record`; kept if `crew_ticket.py activate` already recorded it, and measured against `tickets.baseBranch` when your branches come from `development`), works the plan step by step — test first, watch it fail, implement, watch it pass — and the **plan-approval + scope guard hook** blocks any write outside the spec's Touch globs before it happens, not after. Before the first step it prints the recurring-findings checklist (`recurring_findings.py --ticket T-0091`): the defect classes earlier reviews kept finding on paths like the spec's Touch list, a few probes each, kept open while the plan is worked and handed to whichever developer types.

5–6. Tests and docs. Coverage lands as part of implementing the plan's steps. `/crew:docs` runs next and usually says "none" — most tickets touch no document that needs updating. Then `crew_gitignore.py check` asks whether the ticket brought a language or manifest whose ignore patterns are missing: it adds them only if the spec's Touch covers `.gitignore`, and otherwise the PR body lists them and the next `/crew:onboard` adds them.

6b. The standards self-check. Before the review, the session answers every development standard in the effective set (crew's generic GEN standards, any per-language set a changed file matches, and this repository's `.crew/standards.md` overlay) in `.work/tickets/T-0091/selfcheck.md`: `addressed` with evidence, or `n/a` with a reason. `crew_standards.py stamp` refuses an incomplete answer sheet and binds a complete one to the exact review bundle, so any later edit needs a new stamp.

7. Review. `/crew:implement` calls `/crew:review T-0091` last, after tests, docs and the self-check; without a current stamp the review refuses before it spends a round. Codex reviews the bundle (or Copilot, or the Claude fallback, whichever survives the author-family strike), reports BLOCK/FIX/NIT lines, and you fix the BLOCKs. Two rounds total, ticket-wide — a third is refused

and the ticket becomes `NEEDS_REPLAN`. Under `/crew:autopilot`, a final round with a BLOCK stops for you unless you set `autopilot.maxAutoReplans` (default `0`, off): then autopilot rejects it itself, writes a successor plan that quotes every BLOCK and FIX line, approves it under `autopilot.approval` and reviews again, at most that many successor plans per ticket. Each round's findings go into a `standards-proposals-r<N>.md` file, one row per finding, once the review ledger records that round as CLEAN or FINDINGS (an INCOMPLETE round gets none); you approve or reject each proposed standard, and nothing is added to a standards file on its own.

With parallel lanes in one clone, arm its **merge train** once (`crew_train.py arm`, L-0520). Lanes still implement at the same time, overlapping Touch or not; only gate and land queue. The review round takes the train itself (L-0526; `crew_train.py acquire --ticket T-0091` does the same by hand): a ticket whose Touch overlaps one already holding it waits - `/crew:review` stops with exit 10, colliding paths named, no round spent - and gates next, in the order the lanes reached their gate, while a ticket with a disjoint Touch gates at once. Catch up with `crew_train.py catch-up --ticket T-0091` - a `git merge` of the base, never a rebase, with git rerere on so a conflict resolved once replays next time. A replay is left unstaged and listed: inspect it and `git add` it; the review prompt lists it for the reviewer. crew never turns on `rerere.autoupdate`, and the version files (`plugin.json`, `marketplace.json`, `PLUGINS.md`, `CHANGELOG.md`) are never replayed - they come back conflicted, named as forgotten, for you to resolve by hand.

8. Done. You type `/crew:done T-0091`. Three checks, all required: the review receipt rebuilds clean, the verify gate is clean, and the completion audit (the whole tree diffed against the scope base, not counting a file that is byte-identical to main as last merged) finds nothing outside scope. Any one failing refuses the close and names what to fix. On success it appends a metrics row, marks the ticket done, and clears a stale handoff. The metrics row (schema 1.1) carries tokens split by kind, each assistant message counted once, and a list-price cost. They are an attributed lower bound: only the Claude Code transcripts tied to the ticket by a `Ticket ID: <id>` line, a `/crew:<command> <id>` or its own worktree count (a dispatch prompt's `Ticket ID:` line ties a subagent), and when none can be tied both read `UNKNOWN`, never `0`.

With the train armed, landing is part of done: `crew_train.py check-land --ticket T-0091 --pr <n>` refuses unless T-0091 holds the train, `git merge-tree` against the base is clean, the base has not moved in Touch paths, and HEAD carries the review receipt and a green gate; then it prints `gh pr merge <n> --merge --match-head-commit <sha>` for you to run (the train never merges; only autopilot's ship step does, under `autopilot.ship`; with `autopilot.deploy` set, its deploy phase then names `/crew:promote <env>` for the first GitHub environment, or stops at `deploy-`

`target` / `failed-deploy`). When it refuses, land in this order: `crew_train.py catch-up` (resolve any conflict), bump the version one past the base's, refresh the artifacts, commit, gate the merged head, review it again if `review_ledger.py --check-receipt` reads stale, then `check-land` again, so the tree the gate passed is the tree that lands. After the review, a re-anchor changes only the `anchor:` sha and regenerates the rules; its provenance goes in the ticket's `notes.md`. After the merge, `crew_train.py release --ticket T-0091 --merged <merge sha>` frees the train and tells every overlapping lane to merge the base now.

What Each Hook Does, in Order

When	Hook	Does
Every write	plan-approval + scope guard (`PreToolUse`)	refuses an edit with no approval receipt, or outside the spec's Touch
End of turn	verify gate (`Stop`)	runs the rules for what changed that fit `verify.stopBudgetSeconds`; refuses to end the turn on a red check, never on a deferral (a rule too big for the budget goes to CI); prints one verdict line and the failures, the rest is in `.crew/.verify-gate.log` (`--verbose` shows it)
End of turn	completion scope audit (`Stop`)	diffs the whole tree against the scope base, catches shell-made writes too
Session start	context	brief on branch, open ticket, gate state, codemap freshness

None of these ask you anything mid-turn — a blocked write or a red gate shows up as a refusal with a reason, not a prompt.

Light Path — `/crew:fix`

For a small, well-understood change (one subsystem, no new behaviour, known cause, nothing touching auth/SQL/IaC/secrets/migrations):

```
/crew:fix the changed-file list in implement.md prints nothing on an empty diff
```

Every phase still runs, compressed:

- **Direction:** one line, not a question round — `Fix: <what you typed>`.
- **Spec:** six lines — Intent, Touch, Acceptance — Exclusions, Evidence and Unknowns omitted, not forgotten.
- **Plan:** exactly one step, still shown, still needing the approval receipt.
- **Implement, tests, docs:** same as the full path, sized to one step.
- **Review:** one Codex round instead of two.
- **Done:** the same three checks — the light path does not skip its own gate.

If the change turns out to be bigger than it looked — a second subsystem shows up, the cause wasn't what you thought — the session says so and hands off to `/crew:spec <id>` to fill in what the light path left out, rather than forcing the rest of the ticket through a path that no longer fits it.

Web Testing — `/crew:webtest`

For a ticket whose acceptance criteria describe UI behaviour (the `stack-web` skill triggers on `playwright.config.*`, `@playwright/test`, or an Angular e2e suite), `/crew:webtest` sits inside the normal eight phases rather than replacing them:

1. **Planner.** `playwright-test-planner` reads the ticket's acceptance criteria and writes `specs/<ticket>.md` — what to test, not code yet.
2. **Plan.** crew's own `/crew:plan` reads that spec the same way it reads any other Files/Test/Risk plan, and needs the same approval receipt before implement starts.
3. **Generator.** `/crew:implement` calls `playwright-test-generator` against the approved spec to write the actual `*.spec.ts` files.
4. **Healer.** A failing generated test goes to `playwright-test-healer` for locator/wait repairs only. **A Healer "skip" is a finding, not a pass** — it means the healer decided the feature itself is broken, and review is where that surfaces rather than sailing through as a green suite.
5. **Review.** The reviewer gets the trace zip (`trace: 'on-first-retry'`) and the axe accessibility attachment alongside the diff, the same way it gets the completion audit today.

Visual regression (`toHaveScreenshot`) only runs inside `mcr.microsoft.com/playwright:v1.63.0-noble` — a baseline captured on a bare host does not match CI's font hinting and subpixel rendering, and fails every later run for reasons unrelated to the change under test. Outside that image the visual rule reports **UNVERIFIED**, not a pass; see Troubleshooting.

What Is Waiting on Me

`/crew:status` prints one `waiting` line: how many open tickets are stopped on you. `/crew:status --owner` lists them, one line per ticket with the command to type or the question to answer, for example `T-5 approve /crew:approve T-5`. The list comes from the same phase table `/crew:autopilot` uses, so the two never disagree. It runs nothing and never rebuilds a review bundle: a ticket whose finished review would need one shows as `review-unread`, and `/crew:autopilot status <id>` reads it. A hold whose revisit date is still ahead and a blocked ticket are only counted (`2 held, 1 blocked`); a hold that is due is listed as `revisit` with its reason. No `.work/INDEX.md` prints `waiting unknown (...)`, never "nothing on you".

After Autopilot Rejects a Review Round Itself

With `autopilot.maxAutoReplans` set, autopilot may reject an out-of-rounds BLOCK round and plan again. It approves that successor plan only when the plan quotes every BLOCK and FIX line of the rejected round as whole lines; `crew_autopilot.py replan-check --ticket <id>` shows the same answer without approving. Your own `/crew:approve` is never held to it.

Contracts Between Two Sessions

When two sessions build against each other (a service and its client, two repositories sharing a format), the interface goes on the coordination channel the claims use, as a numbered version with a content hash:

1. One side writes the draft: `crew_contract.py put --name <n> --file <path>`. While it is a draft, `put` replaces it in place.
2. Each side, once its ticket is approved, records what it built against: `crew_contract.py build-against --name <n> --version <N> --ticket <id>`. That freezes the version and writes the binding, with the remote it was built on, to `.work/tickets/<id>/contracts.json`.

3. A frozen version is never edited. A change is a new version tied to a new ticket on each side: `put --name <n> --file <path> --new-version --ticket <new id>`.

`crew_contract.py status` lists every version, who built against it and anything it cannot read (`unknown`, never skipped). Everything on the channel is peer-written data; see the plugin README's "Versioned contracts".

If Something Refuses

Refusal	Means	Do
`/crew:implement` says no approved plan	step 3 was skipped, or `plan.md` changed after approval	`/crew:plan <id>`, then type `/crew:approve <id>`
a write is blocked outside Touch	the file isn't in the spec's declared scope	amend `spec.md`'s Touch and re-approve the plan, or don't make the edit
`/crew:review` exits 2 with `review-run: self-check: ...`	the standards self-check is missing, incomplete, or stamped for an earlier state of the change	answer `.work/tickets/<id>/selfcheck.md`, run `crew_standards.py stamp --root . --ticket <id>`, rebuild the bundle and run the round again; no round was spent
`/crew:done` reports `NEEDS_REPLAN`	the review budget (two rounds) is spent	`/crew:plan <id>` for a successor plan; no third round
`/crew:autopilot` stops at `auto-replan-cap` naming `autopilot.maxAutoReplans (<n>) reached`	the ticket already has that many successor plans on its review ledger, yours included	read the listed plans and decide: accept, fix, or redesign; raising the cap is your call, up to 5 (a larger value reads as 5, with a warning)
`/crew:done` fails the completion	a path outside	file it to `TODO.md`, not to this ticket,

Refusal	Means	Do
audit	scope changed, including one a shell command wrote	then rerun

See Troubleshooting for the rest.

Scope and Approval

Every crew 1.0 ticket has a contract: the spec says which files it may touch, the plan says which of those each step changes, and you approve both. Two hooks then hold the session to that contract. This section explains what they do, what they print, and how to change scope while you work.

The Contract

A ticket lives in `.work/tickets/<id>/` (once done and archived with `crew_tracker.py archive`, in `.work/tickets/Complete/<id>/`; crew finds it in either place, and a ticket present in both is "could not tell", which refuses). An id is `LETTERS-digits` with any prefix (`T-`, `L-`, `W-`):

- `spec.md` has six sections: `## Intent`, `## Exclusions`, `## Evidence`, `## Unknowns`, `## Touch` and `## Acceptance checks`. `## Touch` lists one repo-relative glob or path per bullet:

```
## Touch

- `src/export/**`

- `tests/test_export.py`

- `docs/export.md`
```

- `plan.md` is a list of steps. Each step has a `Files:` line, a `Test:` line and a `Risk:` line. Every `Files:` entry must fall inside Touch. A plan cannot add a path the spec does not list.
- `next.md` is optional, and nothing in crew writes it: you do, by hand. One `key: value` per line says who the ticket waits on and what happens next: `waiting-on:` (`owner`, `agent`, `external` or a ticket id), `next:`, `reason:`, `revisit:` (a `YYYY-MM-DD` date) and `superseded-by:`

(a ticket id). It is not part of the contract: approval does not hash it and `validate` never reads it, so you can edit it at any step without approving again.

Check the contract at any time:

```
python3 <crew>/hooks/scripts/crew_ticket.py validate --ticket T-0042
```

A plan path outside Touch is reported as `INVALID: plan Files entry '...' is outside spec ## Touch`. To fix it, add the path to Touch. crew never widens Touch for you.

Approval

When the plan is ready, the session asks you to approve it. Type this yourself:

```
/crew:approve T-0042
```

crew's UserPromptSubmit hook sees the prompt you typed, validates the contract, then records the sha256 of `spec.md` and `plan.md` in `<git-common-dir>/crew/tickets/T-0042/approval.json`, marked `approved_via: "user-prompt"`. If the contract does not validate, the hook blocks the prompt and says why. That file is outside the worktree, and the scope guard refuses any Write or Edit to it and any shell command that runs `crew_ticket.py approve`, so the session cannot approve its own plan. The one exception is opt-in: with `scope.allowCliApproval: true` and `autopilot.approval` set to `self` (or `risk`, for a `risk: low` spec), `/crew:autopilot` runs `crew_autopilot.py approve`. It writes what every approval route writes: `approval.json` (marked `approved_via: "autopilot"`), `scope-tickets.json` on a ticket's first approval, and, for a distinct successor plan under a spent review budget, the review ledger moved NEEDS_REPLAN -> IN_REVIEW. Both default off, so out of the box nothing self-approves. Inside an `autopilot.sleep.schedule` window, `autopilot.sleep.approval` stands in for `autopilot.approval` (T-0053), and a receipt written asleep stops standing when the window ends if the day value would not have approved it: in the morning that ticket waits for `/crew:approve <id>`. `/crew:autopilot sleep` starts sleep mode now, until the window's end (12 hours with no window), and `/crew:autopilot wake` ends it now (L-0652); they write only `<git-common-dir>/crew/autopilot-sleep.json`. Until L-1504 a manual sleep only tightens: outside the window it applies a night value only where it is stricter than the day value. `sleep` needs `scope.allowCliApproval: true` and an override stricter than its day value, or the window open. Every approval and answer autopilot makes while asleep goes into the local log `.work/autopilot/sleep-log.md` (L-0653; never committed, never read to decide anything), and

the morning summary prints them once, grouped by ticket: `/crew:autopilot wake` prints it, and once the window has ended `settings` warns until `crew_autopilot.py sleep-summary` has reported them. Autopilot's one other ledger writer is `crew_autopilot.py auto-reject` (T-0074), only with `autopilot.maxAutoReplans` set: it moves the ledger REVIEWED -> NEEDS_REPLAN after a final round with a BLOCK. The approval is a step you take, not a lock.

`crew_ticket.py status --ticket T-0042` prints one of three states:

- `approved`: both files match the receipt.
- `stale`: `spec.md` or `plan.md` changed since approval.
- `none`: the ticket has never been approved.

While You Implement: The Scope Guard

Before every Write, Edit, MultiEdit or NotebookEdit, the guard checks the target file:

- The ticket's own `.work/tickets/<id>/` files are always allowed, so you can amend the spec and plan.
- Any other file needs a current approval and a path inside Touch.
- The code map, the diagrams dir, the graph dir and `.claude/rules/` are writable for an approved ticket, because `/crew:implement` step 6 refreshes them.
- `.crew/metrics.md` is writable with or without an approval: `/crew:review` appends its row by hand when the script could not (`crew_ticket.CREW_WRITE_ALLOWED_PATHS`, that one file).
- The scope base, the verify gate's records and marker and `.crew/metrics.jsonl` are refused to Write/Edit in every mode but `off`, even inside Touch: the audit and the review bundle leave them out, so a write to them would be invisible.
- Nothing else is exempt: `.crew/` beyond the code map and the metrics row (`config.json`, `verify.json`, `standards.md`, and files crew writes but also trusts, such as `.crew/tfplan/`, `incident.json` and `.deploy-in-flight`), `TODO.md`, the rest of `.claude/` and crew's own policy files. If the ticket changes one of them, list it in Touch.
- Approval receipts, the review ledger and the scope base are always refused.

The guard checks the real file a symlink points to and the path as written, so a link cannot move a write into or out of scope. `..` is resolved the way the operating system resolves it. Refusals look like this:

```
SCOPE GUARD: refused Edit on other/keep.py.

  Reason: other/keep.py is outside T-0042's spec ## Touch.

  To widen scope: amend .work/tickets/T-0042/spec.md ## Touch (and plan.md), then
ask the user to type

  `/crew:approve T-0042`. (scope.mode is block)
```

At the End of a Turn: The Completion Audit

The guard only sees the editing tools. A `sed -i`, a shell redirect or a formatter can still change files. So when the session stops, the completion audit compares the whole tree with the commit the ticket started from. It covers committed, staged, unstaged and untracked changes, and both ends of a rename. After you merge main, a file byte-identical to the merged main commit is not counted: it is main's change, not the ticket's; the verdict names that commit. If any changed path is outside Touch, the audit blocks the stop and lists the paths in six lines or fewer. It never blocks the continuation it caused. crew's own ticket-flow bookkeeping (the scope base, the gate's records, the metrics files) is never a changed path, whatever `.gitignore` says, so a `.crew/` path the audit lists (say `.crew/verify.json` or a committed `.crew/incident.json`) is a real finding, not crew's noise.

The verify gate also prints an `outside-scope:` line every Stop, in any `scope.mode`. It reads the same approved `spec.md ## Touch` the audit reads and leaves out only what the audit leaves out, so it names the same paths early, a `sed -i` write, `TODO.md` and a tracked `.crew/` file included; it never blocks. `outside-scope: (could not tell - <why>)` means it could not judge: no `spec.md`, no `## Touch`, a Touch not yet approved, a ticket still in the pre-1.0 layout, or an `INDEX.md` open ticket with no `.work/tickets/<id>/` folder.

A changed refresh artifact passes the audit only when a path the ticket changed reaches it and the edit is a re-anchor (the `anchor:` or provenance sha moved forward to a commit on this branch) or a regeneration (`.claude/rules/` as `crew_instructions.py rules` writes them, the graph after a code change). A deleted rendered diagram or graph file, a symlink at or along an artifact's path, a file whose git mode changed, or one removed from the index but left on disk (`git rm --cached`: the commit deletes it) never passes. A map claim edited without a re-anchor is listed with `[anchor did not move]`, and belongs in Touch if that is what the ticket means to do. When git cannot answer, a rule file cannot be read, a short anchor is ambiguous (two commits share it), the artifact dirs cannot be resolved, or a directory the hook cannot search hides whether the config, a rule or a map exists, or two configured artifact dirs are equally specific for

a path, the listing says `[could not tell: ...]`, and that never passes. (These verdicts are `crew_refresh_check.py`'s; the audit applies them once L-0540 lands, and until then admits the artifact dirs for an approved ticket.) `/crew:done` runs the same check:

```
python3 <crew>/hooks/scripts/completion_audit.py --check --ticket T-0042
```

The audit sees what git sees. Gitignored files, including everything under `.crew/`, and `.work/` are outside it.

Amending Scope

1. Edit `spec.md` `## Touch` to add the path. Update `plan.md` if a step now changes it.
2. Run `crew_ticket.py validate --ticket <id>`.
3. Type `/crew:approve <id>` yourself.

Between steps 1 and 3 the approval is stale. The guard refuses edits outside the ticket directory until you approve again.

After Two Review Rounds: A Successor Plan

If a ticket uses both review rounds without an accepted receipt (refunded tool-failure rounds do not count), it moves to `NEEDS_REPLAN`. With `autopilot.maxAutoReplans` at 1 or more, autopilot does this itself for a final round with a BLOCK (`auto-reject`), then writes and approves the successor plan under `autopilot.approval`; the cap counts every successor plan on the ledger. With notify on, the last round carrying a BLOCK sends the `Review out of rounds` ping (`-> /crew:plan <id>`) when `/crew:autopilot` stops on it. Write a different plan and approve it. `approve` reports `review may continue`, and the ledger gives the new plan two fresh rounds. Approving the same plan again is refused and exits with status 3.

Modes

Set `scope.mode` in `.crew/config.json`:

Value	What happens
`off`	Default. Neither hook does anything.
`report`	Everything is allowed. Would-be refusals go to `.crew/guard.log` and appear as a

Value	What happens
	system message.
`block`	Refusals block.
`auto`	`report` for the first ten tickets approved in the repository, then `block`.

If the config file does not parse, or `scope.mode` has an unknown value, crew treats it as `block` and says so.

Which Ticket Is Active

`crew_ticket.py activate --ticket <id>` sets the active ticket for the current worktree. `deactivate` clears it, and `active` prints it. Without a setting, crew uses the open ticket in `.work/INDEX.md` if `.work/tickets/<id>/` exists. With no active ticket, the edit guard only protects approval and ledger state, and the completion audit does nothing.